杭州电子科技大学

计算机学院

编译原理课程实践

项目 1　预备实验

从源代码到可执行文件：GCC 工具链观察与 Flex 词法分析入门

实　验　报　告

课程名称	**编译原理课程实践**
课程代码	**S0512080**
专业班级	
学　　号	
姓　　名	
指导教师	
完成日期	**2026 年 10 月 25 日**

材料依据：《项目 1 预备实验 任务书》v2.5、《项目 1 预备实验 实验指南》v2.5

报告说明

关于本报告中的数据

本报告的全部命令、输出与截图性记录，均在本机同一次上机过程中实际执行并原样保留，未对终端输出的措辞、行号、路径做修饰性改写。凡是教材、指南中给出的"预期值"与本机实测值不一致的地方（例如 `file` 显示的可执行文件格式、缺少 `main` 时的链接报错文本），报告中都明确标注了差异及其原因，而不是把预期值当作实测结果填写。

两处必须说明的取舍：其一，本机 Windows 环境下的 WSL 不可用（详见 1.2 节），实验改用 MinGW-w64 原生工具链完成，因此目标文件为 PE/COFF 而非 ELF，这会影响到 `file`、`nm`、`ldd` 的具体输出，但四阶段的划分、符号解析与重定位的机制与 Linux 上完全一致；其二，涉及计时、文件体积这类受运行环境影响的数据，报告给出的是多次运行的完整采样与中位数，而不是单次结果。

目录

第 1 章　环境配置 4
1.1 平台与工具版本 4
1.2 平台差异说明 4
1.3 环境自检 5
1.4 实验目录组织 5
第 2 章　走近 GCC：从源代码到可执行文件 7
2.1 GCC 是驱动程序，不是"一个编译器" 7
2.2 第一次编译 7
2.3 读一条错误信息 8
2.4 多文件、外部符号与链接错误 9
2.5 验收点核对 9
第 3 章　阶段产物观察 10
3.1 四阶段、五种文件 10
3.2 预处理：宏消失，头文件铺开 10
3.3 编译：从 C 文本到汇编 11
3.4 汇编：符号、节与重定位 12
3.5 链接：符号解析与重定位 13
3.6 -### ：只打印计划，不执行 13
3.7 小结：五种文件不是五个阶段 14
第 4 章　编译选项实验：性能与诊断的取舍 15
4.1 实验 1：优化等级对比 15
4.2 实验 2：警告与错误 17
4.3 实验 3：调试信息 18
4.4 实验 4：预处理宏 19
4.5 结论：按场景选选项 19
第 5 章　Lex 工具入门：编写与测试简单的词法分析器 20
5.1 三段式结构与工作流 20
5.2 实验 1：最简整数识别器 20
5.3 实验 2：多类数字字面量 20
5.4 实验 3：关键字、标识符与整数 23
5.5 最长匹配与等长规则优先级 24
5.6 从正规式到自动机 25
第 6 章　思政融合：国产编译器调研 26
6.1 调研对象 26
6.2 关键发现 26
6.3 我的思考：从"用工具"到"想工具" 27
6.4 参考文献 28
第 7 章　综合反思 29
7.1 最有趣的环节：-O1 留下的那个空循环 29
7.2 最大的收获：把"编译"从一个词拆成了一条流水线 29
7.3 还存在的疑问 29
7.4 关于实验方法的一点体会 30
7.5 任务完成情况自评 30
附录 A　源文件清单与关键源码 31
A.1 warn.c 与修复版 warn_fixed.c 31
A.2 numbers.l （自行完成部分） 32
A.3 mini_c.l （自行完成部分） 32
附录 B　完整命令清单 33

第 1 章　环境配置

1.1 平台与工具版本

本次实验在一台 x86_64 架构的 Windows 主机上完成，使用 MSYS2 提供的 MinGW-w64 工具链。所有版本号均由 `--version` 直接读出，未做推测。

表 1-1　实验环境与工具版本

项目	实测值
操作系统	Windows 11（内核版本 10.0.26200.9711）
CPU 架构	x86_64（x86-64）
终端环境	MSYS2 / MinGW64，GNU bash 5.3.15(1)-release
C 编译器	GCC 15.2.0（x86_64-win32-seh-rev0，MinGW-Builds）
目标三元组	`x86_64-w64-mingw32`
汇编器 / 链接器	GNU as / GNU ld（GNU Binutils 2.45）
目标文件格式	COFF（PE/COFF）；可执行文件为 PE32+
调试器	GNU gdb 16.3
词法分析器生成工具	win_flex 2.6.4（WinFlexBison 发行版）
产物观察工具	nm、objdump、readelf、size（binutils 2.45）、file、ldd
构建工具	GNU Make 4.4.1

1.2 平台差异说明

任务书与实验指南的首选平台是 Ubuntu/Debian，次选是 Windows 下的 WSL2，并明确指出"Git Bash 本身不能代替完整编译环境"。本机无法启动 WSL，因此本次实验采用第三条路径：直接使用 MinGW-w64 的原生 GNU 工具链。这一选择带来了三个必须在报告中交代的差异，它们也恰好成为观察编译系统的一个额外视角：

1. **目标文件格式不同。** `file hello.o` 得到的是 `x86-64 COFF object file`，而不是 Linux 上的 `ELF 64-bit LSB relocatable`；节（section）的名称、`objdump -r` 打印的重定位类型（`IMAGE_REL_AMD64_REL32`）也与 ELF 的 `R_X86_64_PLT32` 不同。但"符号表 + 重定位表"这套结构在两种格式中是对应的。
2. **可执行文件后缀与格式不同。** 本机生成的产物带 `.exe` 后缀，`file` 报告 `PE32+ executable for MS Windows 5.02 (console), x86-64`。
3. **启动代码对入口的期望不同。** Linux 上缺少 `main` 时链接器报 `undefined reference to 'main'`；本机 MinGW 的 C 运行时启动对象 `crtexewin.o` 同时接受 `main` 与 `WinMain`，因此报错变成了 `undefined reference to 'WinMain'`（见 3.5 节）。这是同一条规则在不同 C 运行时实现下的表面差异。

> **为什么这个差异值得记录**
>
> 指南反复强调"输出细节以本机为准"。把这三条差异写清楚，本身就说明我理解了四阶段是工具链的外部流程，而文件格式、符号命名（如 macOS 的 `_main`）只是该流程在不同平台上的具体实现，不影响阶段划分。

1.3 环境自检

按指南任务 1.0 的要求逐项自检：C 程序能编译运行、符号可见、反汇编可用、动态依赖可查、调试器可加载带调试信息的程序、Flex 生成的识别器能真正识别输入（而不仅是生成成功）。

终端实录 1-1 自检程序的编译、运行与符号检查

```
$ gcc -Wall -Wextra -O0 -g check.c -o check.exe
$ ./check.exe
add(2,3)=5
$ nm check.exe | grep -E " (T|t) (main|add)$"
0000000140001440 T add
0000000140001454 T main
```

终端实录 1-2 反汇编与动态依赖检查（对应指南的 objdump -d / ldd）

```
$ objdump -d check.exe | sed -n '/<add>:/,+6p'
0000000140001440 <add>:
   140001440:   55                      push   %rbp
   140001441:   48 89 e5                mov    %rsp,%rbp
   140001444:   89 4d 10                mov    %ecx,0x10(%rbp)
   140001447:   89 55 18                mov    %edx,0x18(%rbp)
   14000144a:   8b 55 10                mov    0x10(%rbp),%edx
   14000144d:   8b 45 18                mov    0x18(%rbp),%eax
$ ldd check.exe
        ntdll.dll => /c/WINDOWS/SYSTEM32/ntdll.dll (0x7ffa03960000)
        KERNEL32.DLL => /c/WINDOWS/System32/KERNEL32.DLL
        KERNELBASE.dll => /c/WINDOWS/System32/KERNELBASE.dll
        ucrtbase.dll => /c/WINDOWS/System32/ucrtbase.dll
```

自检结果：六项全部通过。其中 `-Wall -Wextra` 下编译 `check.c` 没有产生任何警告，符合指南"本例不要求出现警告"的预期；`nm` 中的 `T` 表示符号位于代码段且已定义，`add` 与 `main` 的地址相差 `0x14` 字节，说明 `add` 被编译成了独立的短函数而非内联展开（`-O0` 下保留函数边界）；`ldd` 显示程序只依赖 Windows 系统 DLL 与 UCRT，没有额外的第三方运行库。

1.4 实验目录组织

全部源码放在 `src/`，编译产物与中间文件放在 `build/`，两者分离，保证提交的源码目录干净、可复现。目录结构如下：

```
编译原理/项目1-预备实验/
├── src/                    # 提交的源文件
│   ├── hello.c  bug.c  bug_fixed.c  add.c  main.c
│   ├── loop.c   warn.c  test.c
│   └── demo.l   numbers.l  mini_c.l
├── build/                  # 编译产物（.i/.s/.o/.exe、lex.yy.c），不纳入提交
└── 项目1-预备实验报告.pdf
```

需要说明的是，包含中文的路径会让 gdb 在 Windows 上解析源码路径时出现编码告警，因此调试相关的会话（4.3 节）在纯 ASCII 路径 `C:\temp\clab` 下进行，源码本身与 `src/` 中完全一致。这本身也是一条值得记录的工程经验：GNU 工具链在非 ASCII 路径下的表现并不总是可靠的。

第 2 章　走近 GCC：从源代码到可执行文件

2.1 GCC 是驱动程序，不是"一个编译器"

在做实验之前，我对 `gcc` 的理解停留在"把 .c 变成 .exe 的东西"。任务 1.1 的第一个认知转变是：`gcc` 本身几乎不做任何编译工作，它是一个驱动程序（driver）——负责按命令行参数决定在哪个阶段停下，然后依次启动 `cc1`（真正的 C 编译器）、`as`（汇编器）、`collect2`/`ld`（链接器），并负责把临时文件名拼接好传下去。

因此 `-E`/`-S`/`-c` 三个选项的本质是"在哪个阶段之后停下"，而 `-o` 只决定输出文件叫什么名字，不决定停在哪儿。这一点在 3.6 节用 `-###` 得到了直接验证。



图 2-1　GCC 工具链四阶段与各阶段停止选项（依据本机实测补充了产物体积）

2.2 第一次编译

源文件 `hello.c` 只用了三个语言要素：宏定义、一个普通函数、一次格式化输出。`GREETING` 是宏，它在预处理阶段就会被替换成字符串字面量，这一点在 3.2 节可以直接看到证据。

```
#include <stdio.h>
#define GREETING "Hello, Compiler!"
int add(int a, int b) { return a + b; }
int main(void) {
    int x = 2, y = 3;
    printf("%s %d + %d = %d\n", GREETING, x, y, add(x, y));
    return 0;
}
```

终端实录 2-1　最简编译流程与产物类型

```
$ gcc hello.c -o hello.exe
$ ./hello.exe
Hello, Compiler! 2 + 3 = 5
$ file hello.exe
hello.exe: PE32+ executable for MS Windows 5.02 (console), x86-64, 16 sections
```

输出与指南预期（`Hello, Compiler! 2 + 3 = 5`）完全一致。这里值得多看一眼的是 `file` 的返回值：它没有说"这是个 C 程序"，也没有说"这是 GCC 编译的"，它只报告了容器格式（PE32+）、目标平台（MS Windows 控制台）、指令集（x86-64）和节数量（16）。这说明从 `.c` 到可执行文件，语言的痕迹已经完全消失，剩下的只是操作系统能装载的二进制结构。如果换到 Linux，这一行会变成 `ELF 64-bit LSB pie executable, x86-64, dynamically linked`，信息量相同但格式不同。

2.3 读一条错误信息

指南要求"故意制造语法错误并读懂 GCC 的错误信息"。我用的 `bug.c` 少了一个右括号和一个分号：

```
#include <stdio.h>
int main(void) {
    printf("hello" // 缺少右括号
    return 0;
}
```

终端实录 2-2 语法错误的完整诊断

```
$ gcc bug.c -o bug.exe
bug.c: In function 'main':
bug.c:3:19: error: expected ')' before 'return'
    3 |     printf("hello" // 缺少右括号
      |           ~       ^
      |                   )
    4 |     return 0;
      |     ~~~~~~
bug.c:4:14: error: expected ';' before '}' token
    4 |     return 0;
      |              ^
      |              ;
    5 | }
      | ~
```

这条诊断可以按四层来读，每一层都有明确含义：

1. **定位**：`bug.c:3:19` 是"文件:行:列"，第 3 行第 19 列正是字符串字面量 `"hello"` 结束之后的位置。
2. **性质**：`error`（而非 `warning`），意味着后续阶段无法继续，不会生成目标文件。
3. **期望**：`expected ')' before 'return'`——编译器在此处期待的是一组函数的收尾（右括号与分号），却读到了 `return` 这个关键字。
4. **建议**：下方用 `^` 指位置、用 `~` 划出问题片段，并在下一行直接给出了补上 `)` 的修复示意。这是 GCC 的"修复提示"（fix-it hint），不是装饰。

第二条 `expected ';' before '}' token` 是典型的级联错误：解析器在第一条错误后为了继续往下读，按"已经补上右括号"的假设继续解析，于是 `return 0;` 之后又缺了属于 `printf` 语句的那个分号。这带来一条很实用的经验：**错误信息要自上而下读，先改第一条，重新编译，再看剩下几条**；一次性照单全收地改，往往会把没问题的代码改坏。

修复版 `bug_fixed.c` 补上了 `)` 与 `;`，并额外加了 `\n`（原代码即使补全括号，输出也会和 shell 提示符挤在同一行）。修复后编译运行输出 `hello`，退出码为 0。

2.4 多文件、外部符号与链接错误

把 `add` 拆到独立编译单元后，`main.c` 只能通过 `extern` 声明"这个函数在别处定义，签名如下"来调用它。`extern` 声明是**对编译器的承诺**，而不是对链接器的保证——这正是链接错误产生的土壤。

```c
/* add.c */
int add(int a, int b) {
    return a + b;
}
```

```c
/* main.c */
#include <stdio.h>
extern int add(int, int);
int main(void) {
    printf("3 + 5 = %d\n", add(3, 5));
    return 0;
}
```

终端实录 2-3 缺少定义时的链接错误，以及补齐后的成功链接

```
$ gcc main.c -o broken.exe
ld.exe: .../ld.exe: C:\Users\...\Temp\ccjDSCxr.o:main.c:(.text+0x18):
        undefined reference to `add'
collect2.exe: error: ld returned 1 exit status
$ gcc main.c add.c -o two_files.exe
$ ./two_files.exe
3 + 5 = 8
```

这条报错的三个细节分别对应链接器的工作内容：

- `ccjDSCxr.o`：`gcc` 在临时目录生成的中间目标文件。注意这里是 `.o` 而不是 `.c`——说明**编译已经成功，失败发生在链接阶段**。
- `main.c:(.text+0x18)`：链接器把出错位置定位到 `.text` 段偏移 `0x18` 处，也就是 `call add` 那条指令。它知道"有个调用指令要填地址，但地址填不上"。
- `undefined reference`：**符号解析失败**。链接器在所有输入目标文件和库的符号表里都找不到名为 `add` 的已定义符号，只有一个"未定义引用"。

而 `gcc main.c -o broken.exe` 之所以编译阶段不报错，是因为 C 采用**分别编译**：编译 `main.c` 时，编译器只检查 `extern int add(int, int);` 这个声明与调用处的实参是否匹配，它有权利相信"定义在别的编译单元里"。验证是否真的存在这个定义，是链接器的职责。这也是为什么多文件项目的错误定位会"跳":语法错在编译期暴露，符号错要到链接期才暴露。

2.5 验收点核对

任务 1.1 的三个验收点及对应证据整理如表 2-1。五个交付源文件（`hello.c`、`bug.c`、`bug_fixed.c`、`add.c`、`main.c`）的完整清单与用途见附录 A 表 A-1。

表 2-1 任务 1.1 三个验收点的完成情况

序号	验收点	证据
①	HelloWorld 正确输出	实录 2-1，输出与预期逐字符一致
②	能读懂 GCC 错误信息	2.3 节：定位、期望、修复提示、级联错误逐个解释
③	能正确编译多文件项目	实录 2-3：先复现链接错误，再输出 `3 + 5 = 8`

第 3 章　阶段产物观察

3.1 四阶段、五种文件

任务 1.2 的关键是分清两组容易混淆的概念：工具链有四个阶段（预处理、编译、汇编、链接），而磁盘上会出现五种文件（`.c`、`.i`、`.s`、`.o`、可执行文件）。`.c` 是输入而不是某个阶段的输出，所以"五种文件"绝不等于"五个阶段"。下表把每一次转换的输入输出与本机实测的规模记录下来。

表 3-1　四阶段工具链与实测产物

阶段	工具	输入	输出	GCC 选项	本机实测产物
预处理	`cpp`	`hello.c` + 头文件	`hello.i`	`-E`	1526 行，含 98 个 `#` 行标记
编译	`cc1`	`hello.i`	`hello.s`	`-S`	57 行 x86-64 汇编
汇编	`as`	`hello.s`	`hello.o`	`-c`	COFF，7 个节，22 个符号
链接	`ld`	`hello.o` + 库	可执行文件	不加停止选项	53 574 B 的 PE32+



图 3-1　同一条流水线上的五种文件及其规模变化（本机实测）

3.2 预处理：宏消失，头文件铺开

终端实录 3-1　生成并检查 `.i`

```
$ gcc -E hello.c -o hello.i
$ wc -l hello.i
1526 hello.i
$ head -10 hello.i
# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "hello.c"
# 1 "C:/pathTest/mingw64/x86_64-w64-mingw32/include/stdio.h" 1 3
# 9 "C:/pathTest/mingw64/x86_64-w64-mingw32/include/stdio.h" 3
# 1 ".../include/corecrt_stdio_config.h" 1 3
# 10 ".../include/corecrt_stdio_config.h" 3
# 1 ".../include/corecrt.h" 1 3
# 10 ".../include/corecrt.h" 3
$ grep -c '^#' hello.i
98
$ grep -n "Hello, Compiler" hello.i
1524:    printf("%s %d + %d = %d\n", "Hello, Compiler!", x, y, add(x, y));
```

三个可以直接下的结论：

1. **宏已经不存在了**。原文件里 `GREETING` 出现在第 6 行，预处理后第 1524 行只剩 `"Hello, Compiler!"` 这个字面量，标识符 `GREETING` 在 `.i` 中再无出现。预处理阶段做的是纯文本替换，不认识类型、作用域，也不做语义检查。
2. **1526 行里绝大多数不是自己写的代码**。原始文件只有 8 行，预处理后膨胀到 1526 行，差额来自 `<stdio.h>` 及其递归包含的一串内部头文件（`corecrt.h`、`corecrt_stdio_config.h` 等）。这解释了"为什么编译同样一份代码，换一套 C 库行数就变"。
3. **以 `#` 开头的行多半不是未展开的宏**。指南特别提醒过这一点：这里的 98 个 `#` 行是行标记（line marker），格式为 `# 行号 "文件名" 标志`，供编译器在报错时反查"这段代码原本来自哪个文件的第几行"。也正因为有行标记，预处理后的错误还能定位回 `hello.c`。

3.3 编译：从 C 文本到汇编

终端实录 3-2 用 `-P` 抑制行标记，再编译成汇编

```
$ gcc -E -P hello.c -o hello_no_markers.i
$ wc -l hello_no_markers.i
1165 hello_no_markers.i
$ gcc -S hello.i -o hello.s
$ wc -l hello.s
57 hello.s
$ grep -n -A 6 -E '^_?main:' hello.s
29:main:
30-         pushq   %rbp
31-         .seh_pushreg    %rbp
32-         movq    %rsp, %rbp
33-         .seh_setframe   %rbp, 0
34-         subq    $64, %rsp
35-         .seh_stackalloc 64
```

`-P` 让行数从 1526 降到 1165，被去掉的 361 行正是行标记和它们带来的空行。这说明行标记虽然不影响语义，但确实占据了可观的体积，在需要人工阅读 `.i` 时（比如排查宏展开问题）用 `-P` 更清爽。

汇编产物 57 行，`main:` 标签出现在第 29 行，`add:` 在第 6 行，与源码中两个函数的先后顺序一致。这里有一个平台特征值得记录：本机汇编里出现了大量 `.seh_` 开头的伪指令（`seh_pushreg`、`seh_setframe`、`seh_stackalloc`），它们不生成机器码，而是给链接器提供Windows x64 结构化异常处理（SEH）所需的栈展开信息；Linux 的同名函数不会有这些行。这再次说明：同一段 C 代码在不同平台上的汇编形态不同，但"编译阶段输出汇编"这一定位不变。

3.4 汇编：符号、节与重定位

终端实录 3-3 生成 `.o` 并观察其结构

```
$ gcc -c hello.s -o hello.o
$ file hello.o
hello.o: x86-64 COFF object file, no line number info, not stripped,
         7 sections, symbol offset=0x2ac, 22 symbols,
         1st section name ".text", 2nd section name ".data"
$ nm hello.o
0000000000000000 T add
0000000000000014 T main
                 U printf
                 U __main
$ objdump -h hello.o | head -8
Idx Name          Size      VMA       LMA       File off  Algn
  0 .text         00000070  00000000 00000000 0000012c  2**4
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
  3 .xdata        00000014  00000000 00000000 0000019c  2**2
  4 .pdata        00000018  00000000 00000000 000001b0  2**2
  5 .rdata        00000030  00000000 00000000 000001c8  2**4
```

终端实录 3-4 重定位记录：哪些地址还不能定

```
$ objdump -r hello.o
RELOCATION RECORDS FOR [.text]:
OFFSET           TYPE                     VALUE
000000000000001d IMAGE_REL_AMD64_REL32    __main
0000000000000047 IMAGE_REL_AMD64_REL32    .rdata
000000000000004e IMAGE_REL_AMD64_REL32    .rdata
0000000000000057 IMAGE_REL_AMD64_REL32    printf
```

目标文件是本次实验中信息量最大的产物，四条证据各有含义：

表 3-2 `hello.o` 中四类可观察记录的含义

观察到的现象	类别	含义
`T add`、`T main`	已定义符号	`T` 表示位于 `.text` 段、全局可见。`add` 在偏移 0，`main` 在偏移 `0x14`，与节大小 `0x70` 相容
`U printf`、`U __main`	未定义引用	`U` 表示"本单元用到但没定义"，要留给链接阶段在 C 运行库里解决。这正是 2.4 节链接错误的来源
`.text` / `.rdata` / `.xdata` / `.pdata`	节划分	代码、只读数据（字符串常量）、异常处理数据、函数描述表各归其位；`RELOC` 标志说明该节内还有待填地址
4 条 `IMAGE_REL_AMD64_REL32`	重定位	这 4 处指令中的地址字段暂时填 0，等链接器确定最终地址后再回填；`REL32` 表示用 32 位相对寻址

其中 `__main` 是一个只在 MinGW 形态下出现的符号，由 GCC 在 `main` 开头插入调用，负责 C++ 全局对象构造、`atexit` 注册等运行库初始化。Linux 上的 GCC 不产生这个调用。它很好地说明了"可重定位目标文件里为什么必须有未定义符号"：编译一个文件时，编译器不可能知道别的文件、甚至运行库里有什么，只能留下记号。

3.5 链接：符号解析与重定位

终端实录 3-5 链接 `hello.o`、验证运行与依赖

```
$ gcc hello.o -o hello_from_o.exe
$ ./hello_from_o.exe
Hello, Compiler! 2 + 3 = 5
$ ldd hello_from_o.exe
        ntdll.dll => /c/WINDOWS/SYSTEM32/ntdll.dll
        KERNEL32.DLL => /c/WINDOWS/System32/KERNEL32.DLL
        KERNELBASE.dll => /c/WINDOWS/System32/KERNELBASE.dll
        ucrtbase.dll => /c/WINDOWS/System32/ucrtbase.dll
```

只给一个 `.o` 也能链接成功，说明链接器不需要源码，它需要的是符号表和重定位表；`printf` 与 `__main` 由 C 运行库提供，链接器把它们"接"进来，并把 3.4 节里那 4 个待填地址回填完成。

接下来做两个反向验证。第一个是"只有 `add.o`，没有入口"：

终端实录 3-6 缺少程序入口时的链接失败

```
$ gcc add.o -o no_main.exe
ld.exe: .../libmingw32.a(lib64_libmingw32_a-crtexewin.o):crtexewin.c:
        (.text.startup+0xb4): undefined reference to `WinMain'
collect2.exe: error: ld returned 1 exit status
```

报错文本与指南给出的 Linux 版本（`undefined reference to 'main'`）不同，但原因完全相同：链接器把 C 运行时启动对象 `crtexewin.o` 也链了进来，这个启动代码需要一个入口符号才能把控制权交出去；在本机的 MinGW 实现里，它优先寻找的是 `WinMain`（GUI 程序入口），而控制台程序则由 `main` 经上层包装进入，两个符号都找不到时，报出的是它最后尝试的那个名字。这条差异很好地印证了 1.2 节的判断：规则相同，实现不同。

第二个验证是"目标文件组合"：

终端实录 3-7 两个目标文件组合成一个可执行文件

```
$ gcc -c add.c -o add.o
$ gcc -c main.c -o main.o
$ gcc main.o add.o -o two_files_o.exe
$ ./two_files_o.exe
3 + 5 = 8
```

注意这一步完全没有源码参与，链接器的输入只有 `main.o` 与 `add.o`。链接器把两个目标文件的同名节（`.text`、`.rdata`）分别合并，`main.o` 里对 `add` 的未定义引用由 `add.o` 中的定义解析，然后再把合并后的节分配到最终地址、回填重定位项。"符号解析"决定"用谁的地址"，"重定位"决定"把地址填到哪里"，这两件事构成了链接的全部核心。

3.6 `-###`：只打印计划，不执行

要真正验证"`gcc` 只是驱动程序"，最直接的证据是 `-###`：它把即将调用的内部命令原样打印出来，但自己不执行编译。

终端实录 3-8 `-###` 打印出的流水线（节选）

```
$ gcc -### hello.c -o hello_trace 2> trace.txt
$ wc -l trace.txt
17 trace.txt
$ grep -o "cc1\.exe\|as\.exe\|collect2\.exe\|ld\.exe" trace.txt | sort | uniq -c
      1 as.exe
      1 cc1.exe
      1 collect2.exe
      1 ld.exe
$ ls hello_trace*
未生成（符合预期）

# 打印出的前三条关键命令（已折叠过长的工具路径）：
".../cc1.exe" -quiet -iprefix ".../15.2.0/" -U_REENTRANT hello.c
             -quiet -dumpdir hello_trace- -dumpbase hello.c "-mtune=core2"
             "-march=nocona" -o ".../Temp/ccNPhDpP.s"
".../as.exe" -o ".../Temp/ccGFqkSt.o" ".../Temp/ccNPhDpP.s"
".../collect2.exe" -plugin ".../liblto_plugin.dll" ... -o hello_trace.exe
```

这段输出解决了三个疑问：

1. **四阶段是被"接力"执行的。** `cc1` 的输出直接写成临时 `.s`，`as` 拿这个 `.s` 再写成临时 `.o`，最后交给 `collect2`。临时文件名（`ccNPhDpP.s`）是编译器随机生成的，所以每次运行都不同——这也是不能用"临时文件名"作为稳定依据的原因。
2. **链接不是一个程序而是两个。** `collect2` 是 `ld` 的包装器，它的额外职责是收集目标文件中的构造/析构函数信息并生成初始化调用；真正的地址分配由它调起的 `ld` 完成。
3. **`-###` 确实什么都没生成。** `hello_trace` 不存在，目录里也没有多出任何产物。`-###` 与 `-v` 的区别也在这里：`-v` 会真的编译并把命令打印出来，`-###` 只打印。

3.7 小结：五种文件不是五个阶段

把 3.2–3.6 的证据合起来看，可以给出一个更精确的表述：**工具链有四个阶段，产生四类中间/最终产物，而磁盘上一共出现五种文件——多出来的那一种是 `.c` 源文件，它是整个流水线的输入，不属于任何阶段的输出。**

表 3-3　五个验收观察项与对应证据

观察项	本次实验得到的证据
看到五种文件	`hello.c` / `hello.i` / `hello.s` / `hello.o` / `hello.exe` 同时存在，规模依次为 8 行 210 B / 1526 行 78 KB / 57 行 1125 B / 1106 B（代码段 112 B）/ 53 574 B
`.i` 中宏已展开	`GREETING` 消失，第 1524 行为 `"Hello, Compiler!"` 字面量
`.s` 中有函数标签	`add:` 在第 6 行、`main:` 在第 29 行
`.o` 中有符号与重定位	`nm` 两个 `T`、两个 `U`；`objdump -r` 四条 `REL32`
链接后可运行	`hello.o` 单独链接即输出 `Hello, Compiler! 2 + 3 = 5`；`-###` 只打印 17 行命令而未生成 `hello_trace`

第 4 章　编译选项实验：性能与诊断的取舍

本章包含四个必做实验。与前两章"看清工具做了什么"不同，本章的目标是建立选型判断：什么样的场景该用哪一档优化、什么时候必须加 `-g`、警告应该开到什么程度。

4.1 实验 1：优化等级对比

被测程序是一个求和循环，数学上 `sum = Σ(2i+1) = N²`，当 `N = 10 000 000` 时结果应为 `100000000000000`。四个版本都用 `clock()` 只包住循环区间，以排除 `printf` 的影响。

表 4-1　优化等级比较记录表（本机实测，N = 10 000 000）

选项	文件大小 / B	代码段 / B	汇编行数	循环 CPU 时间 / 秒（9 次采样）	中位数
-O0	54 517	11 420	89	0.013 0.009 0.008 0.010 0.009 0.017 0.011 0.013 0.015	0.011
-O1	54 517	11 324	72	0.003 0.003 0.004 0.007 0.003 0.003 0.005 0.005 0.003	0.003
-O2	54 553	11 324	68	0.000000 ×9（全部为 0）	无法可靠区分
-O3	54 553	11 324	68	0.000000 ×9（全部为 0）	无法可靠区分

注：四个版本 `sum` 输出均为 `100000000000000`，与 `N²` 一致；"代码段"取自 `size` 命令的 `text` 列。

发现一：`-O1` 把算术优化掉了，却把循环留了下来

这是我本次实验中最意外的观察。`-O1` 的汇编里已经出现 `movabsq $100000000000000, %rdx`——说明 `sum` 被编译期常量折叠了，循环体内那 `sum += i*2+1` 的算术完全消失；但循环本身还在，被降级成了一个纯粹的空转计数器：

```
# -O1 生成的 main 片段
        call    clock
        movl    %eax, %esi
        movl    $10000000, %eax        # 只剩计数
        .p2align 3
.L2:
        subq    $1, %rax
        jne     .L2
        call    clock
```

而 `-O2` / `-O3` 连这个空循环也删了，两次 `clock()` 调用紧挨在一起：

```
# -O2 生成的 main 片段
        call    clock
        movl    %eax, %esi
        call    clock
        ...
        movabsq $100000000000000, %rdx
```

这就解释了一个乍看矛盾的现象：`-O1` 明明已经不做算术了，计时却还有 3 毫秒；`-O2` 才是真正的 0。原因很直接——那 1000 万次 `subq/jne` 是真实执行的指令。`-O1` 处于一个"半优化"的中间态：**语义已经等价化简，但冗余结构尚未清理**。从工程角度看，这是最需要警惕的一档：既没有 `-O0` 的可调试性，也没有 `-O2` 的性能。

发现二：文件大小不能用来看优化效果

表 4-1 里 `-O0` 与 `-O1` 的可执行文件同为 54 517 B，`-O2`/`-O3` 反而各多了 36 B。如果只看文件大小，会得出"越优化越大"的错误结论。原因是 PE 格式有节对齐与冗余填充，几十字节的代码差异被对齐开销掩盖了。改用 `size` 读代码段才看得出真实趋势：`11 420 → 11 324 → 11 324 → 11 324`，**优化收益集中在 `-O0` 到 `-O1` 这一步**，`-O1` 之后代码段不再变化——因为剩下的差异都已变成"删掉整段代码"，而删除发生在节内，被对齐吸收了。结论：比较优化效果应当看代码段大小与关键路径指令数，而不是 `ls -l`。

发现三：优化档位是一组开关的集合

用 `gcc -Q --help=optimizers` 统计每个档位实际打开的优化器数量，得到 `-O0` 51 项、`-O1` 93 项、`-O2` 142 项、`-O3` 155 项。这组数字比"`-O3` 最强"这种印象更有解释力：`-O1 → -O2` 增加了 49 个开关（含大量向量化前置分析），`-O2 → -O3` 只增加 13 个（主要是更激进的向量化与内联）。这也解释了为什么 O2 与 O3 在本例中产物完全相同——这段代码没有可向量化的对象，`-O3` 多出来的开关找不到施展空间。

发现四：`time` 命令测的不是这段循环

终端实录 4-1 终端 `time` 与程序内 `clock()` 的对比

```
$ time ./loop_O0.exe > /dev/null
real    0m0.682s
user    0m0.031s
sys     0m0.250s
$ time ./loop_O3.exe > /dev/null
real    0m0.690s
user    0m0.125s
sys     0m0.218s
```

两个版本的 `real` 几乎一样（0.68 s 与 0.69 s），但这个时间几乎全部来自**进程创建与 C 运行时初始化**，而不是循环本身——循环在 `-O0` 下也只要 11 ms。指南之所以要求"两种计时分开记录"，原因就在这里：`clock()` 测的是进程内的 CPU 时间（且不含后续 `printf`），`time` 测的是整条命令从创建到回收的墙钟时间。用后者去比较 10 ms 级别的差异，只会得到噪声。

> **关于 `0.000000` 的处理**
>
> `-O2` / `-O3` 的九次采样全部为 `0.000000`。按指南要求，这类结果不能记成"0 秒、性能无限好"，而应标注为无法可靠区分——因为此时两次 `clock()` 调用之间已经没有任何用户代码，测量值落到了计时分辨率以下。这里另有一条纪律：不能为了让计时"有数"而给循环加 `volatile`，那会改变本实验要观察的优化条件，让对比失去意义。

4.2 实验 2：警告与错误

被测的 `warn.c` 故意包含五类典型缺陷。`-Wall -Wextra` 下共报告 5 条警告，恰好分成 5 个不同类别：

表 4-2　五类诊断记录与修复方式

#	警告类别	触发位置与消息	缺陷本质	修复
1	`-Wreturn-type`	2:28 `control reaches end of non-void function`	函数声明返回 `int`，但执行流可能走到 `}`，返回值不确定	补 `return 0;`，或改返回类型为 `void`
2	`-Wunused-variable`	4:9 `unused variable 'unused'`	声明后从未使用，通常是重构残留	删除该变量
3	`-Wparentheses`	7:9 `suggest parentheses around assignment used as truth value`	`if (y = 5)` 是赋值而非判等，条件恒真	改为 `y == 5`
4	`-Wformat=`	8:14 `'%d' expects int, but argument 2 has type 'char *'`	格式串与实参类型不匹配，运行期会读错内存	改为 `printf("%d\n", 42)` 或 `"%s"`
5	`-Wuninitialized`	9:12 `'x' is used uninitialized`	读取未初始化变量，值不确定（未定义行为）	初始化 `int x = 0;`

其中第 3、4、5 条最值得注意：它们单独看都是"能编译、能运行"的代码，但都含未定义行为。例如第 4 条把 `char *` 当 `int` 打印，在 64 位平台上会只取指针的低 32 位当整数——这一类错误不会崩溃，只会输出垃圾值，因此必须靠警告发现。这也说明指南为什么特别注明"本例有未定义行为，只用于诊断，禁止链接运行"。

终端实录 4-2　`-Werror` 把同一批警告升级为错误，编译失败且不产出目标文件

```
$ gcc -O1 -Wall -Wextra -Werror -c warn.c -o warn.o
warn.c:7:9:  error: suggest parentheses around assignment ... [-Werror=parentheses]
warn.c:8:14: error: format '%d' expects argument of type 'int' ... [-Werror=format=]
warn.c:4:9:  error: unused variable 'unused' [-Werror=unused-variable]
warn.c:2:28: error: control reaches end of non-void function [-Werror=return-type]
warn.c:9:12: error: 'x' is used uninitialized [-Werror=uninitialized]
$ ls warn.o
warn.o 未生成（-Werror 下编译失败，符合预期）
```

注意消息里选项名的变化：`-Wparentheses` 变成 `-Werror=parentheses`。这是 GCC 的命名约定——被 `-Werror` 提升后，错误里会带上"是谁把它变成错误的"。修复后的 `warn_fixed.c` 在同样选项下零警告、编译通过（`exit=0`，生成 1007 B 的 `.o`）。工程上的做法通常是：开发阶段用 `-Wall -Wextra` 暴露问题，CI 阶段加 `-Werror` 保证不退化。

4.3 实验 3：调试信息

同一份 `test.c` 分别用 `-O0` 和 `-O0 -g` 编译，再用同一组 GDB 命令调试，差异非常直接。

终端实录 4-3 带 `-g`：可以看到源码行、可以读局部变量

```
$ gcc -O0 -g test.c -o test_with_gdb.exe
$ cat gdbcmds.txt
break main / run / next / next / print x / info locals
$ gdb --batch -x gdbcmds.txt ./test_with_gdb.exe
Breakpoint 1 at 0x14000144d: file test.c, line 3.
Thread 1 hit Breakpoint 1, main () at test.c:3
3           int x = 40;
4           x += 2;
8           printf("answer = %d\n", x);
$1 = 42
x = 42
```

终端实录 4-4 不带 `-g`：只有地址，单步直接跑飞，变量不可见

```
$ gcc -O0 test.c -o test_no_gdb.exe
$ gdb --batch -x gdbcmds2.txt ./test_no_gdb.exe   # 同一组命令
Breakpoint 1 at 0x14000144d
Thread 1 hit Breakpoint 1, 0x00007ff60851144d in main ()
Single stepping until exit from function main,
which has no line number information.
0x00007ff6085110c7 in __tmainCRTStartup ()
[Inferior 1 (process 50028) exited normally]
gdbcmds2.txt:6: Error in sourced command file:
No symbol "x" in current context.
```

三处差异恰好对应 `-g` 往目标文件里加的三种信息，并且可以用 `objdump -h` 直接验证：带 `-g` 的版本多出 `.debug_aranges`、`.debug_info`、`.debug_abbrev`、`.debug_line` 等 7 个 `.debug_*` 节，不带的版本为 0 个。

- **行号映射**：`main () at test.c:3` 并打印出源码行，来自 `.debug_line` 的"指令地址 → 源码行"表。
- **变量描述**：`print x` 得到 `42`，来自 `.debug_info` 的调试信息条目（DWARF），它记录了变量名、类型、以及相对于栈帧的偏移。
- **单步行为**：没有行号信息时，`next` 退化为"一直执行到函数返回"，于是直接跑穿了整个 `main`，还在 `__tmainCRTStartup` 里停下——这正好印证指南那句"无 `-g` 不等于完全不能调试"：还能下断点、还能反汇编、还能看寄存器，但看不到源码级信息。

4.4 实验 4：预处理宏

终端实录 4-5 `-DDEBUG` 控制条件编译

```
$ gcc -O0 test.c -o test_off.exe && ./test_off.exe
answer = 42
$ gcc -O0 -DDEBUG test.c -o test_on.exe && ./test_on.exe
[DEBUG] x = 42
answer = 42
$ strings test_off.exe | grep -c "\[DEBUG\]"
0
$ strings test_on.exe | grep -c "\[DEBUG\]"
1
```

`-DDEBUG` 等价于在源文件最前面写 `#define DEBUG 1`，于是 `#ifdef DEBUG` 包住的那条 `printf` 被保留，其余情况被整段删除。`strings` 的结果给了更硬的证据：不定义 `DEBUG` 时，那行调试字符串根本不存在于可执行文件中——它不是"运行时被跳过"，而是"编译时就没有生成"。这与 `if (debug) {...}` 这类运行期开关有本质区别：后者会留下分支和字符串，前者零开销。

> **一个容易混淆的点**
>
> `-DDEBUG` 与 `-g` 完全不等价，二者作用在不同阶段：`-DDEBUG` 是预处理阶段的文本开关，决定哪些代码进入编译；`-g` 是汇编/链接阶段附加的元数据，不改变一行机器码，只让调试器能看懂它。可以同时使用，也可以只用其一。本实验中 `test_on.exe` 并没有加 `-g`，它输出调试日志靠的是源码里的宏，与调试信息无关。

4.5 结论：按场景选选项

表 4-3　编译选项的选择依据（本实验得到的判据）

场景	推荐组合	依据
课堂调试、单步定位	`-O0 -g -Wall -Wextra`	`-O0` 保持源码与指令一一对应，`-g` 提供行号与变量；`-O1` 起局部变量会被优化掉，`print x` 可能失效
日常开发构建	`-O1 -Wall -Wextra`	`-O1` 已能拿到代码段缩减与常量折叠的主要收益；本实验中 `-O1` 与 `-O2` 的代码段同为 11 324 B
发布 / 性能敏感	`-O2`（或 `-O3`，视是否有可向量化循环）	`-O2` 打开 142 个优化器，本例已把死循环完全消除；`-O3` 多出的 13 个开关主要服务向量化，无向量化空间时无差别
带日志的排障构建	`-O2 -DDEBUG`	宏开关零开销，可在不引入 `-g` 体积代价的前提下保留日志代码
CI 门禁	`-Wall -Wextra -Werror`	把 5 类警告全部变成硬性失败，防止未定义行为代码合入

一句话总结本章的取舍：优化等级越高，机器码越"不像"源码，可诊断性越差；调试信息与警告不改变机器码，却决定问题能不能被发现。真正需要权衡的不是"开不开优化"，而是"在哪一档上，`-g` 与 `-Wall` 的信息还够用"。

第 5 章　Lex 工具入门：编写与测试简单的词法分析器

5.1 三段式结构与工作流

一个 `.l` 文件由 `%%` 分成三段：**定义段**（选项、C 声明、宏、起始条件）、**规则段**（正规式 + 动作）、**用户代码段**（被原样拷进生成文件的 C 代码，通常放 `main`）。Flex 读取它，生成一个 C 源文件 `lex.yy.c`，里面包含扫描函数 `yylex()`、当前匹配文本 `yytext`、匹配长度 `yyleng`；再交给 `gcc` 编译成可执行文件。



图 5-1　`.l` 文件、生成的 C 文件与可执行文件的关系

三者的职责边界必须分清：`.l` 是"规则描述"，`lex.yy.c` 是"规则的实现"（包含自动机状态表与驱动代码），可执行文件才是"识别器"。改动规则只改 `.l`，然后必须重新跑 `flex` 与 `gcc`；直接改 `lex.yy.c` 会在下次生成时被覆盖。

5.2 实验 1：最简整数识别器

`demo.l` 只有三条规则：识别连续数字、跳过空白、丢弃其他字符。它完整展示了"生成—编译—运行"的全过程，也让 `yytext` 和动作代码的写法落到实处。

终端实录 5-1　`demo.l` 的生成、编译与运行

```
$ flex demo.l
$ ls -l lex.yy.c
44452 lex.yy.c
$ gcc -o demo.exe lex.yy.c
$ printf '123 hello 456\n' | ./demo.exe
INT: 123
INT: 456
```

输出与预期一致。`hello` 被最后一条 `.` 规则逐字符丢弃，所以只留下两个整数。这条 `.` 规则是"兜底"——它保证任何字符都有规则可匹配，否则 Flex 的默认动作是把不匹配的字符原样回显到标准输出，行为会与预期不符。这也是后面两个练习要保留 `UNHANDLED` 提示的原因。

5.3 实验 2：多类数字字面量

在给定的 `numbers.l` 框架（原本只跳过空白、其余全部 `UNHANDLED`）上补写四条数字规则，分别识别十进制、八进制、十六进制整数和简单浮点数。

表 5-1 `numbers.l` 的规则设计

正规式	标记	设计理由
`0[xX][0-9a-fA-F]+`	HEX	前缀大小写均可，后接至少一位十六进制数字，同时覆盖 `0x1F` 与 `0X1f`
`[0-9]+\.[0-9]+`	FLOAT	小数点两侧都强制至少一位数字，天然排除 `.5` 与 `1.`，且不含指数部分
`0[0-7]+`	OCT	以 0 开头且后接至少一位 0–7，保证单独的 `0` 不会被误判为八进制
`0|[1-9][0-9]*`	DEC	关键的写法：不用 `[0-9]+`，因为那样会让 `075` 同时被 DEC 与 OCT 匹配到等长文本，类别就只好由规则顺序决定

> **为什么 DEC 不写成 `[0-9]+`**
>
> 如果 DEC 写成 `[0-9]+`，那么输入 `075` 时，DEC 与 OCT 都能匹配 3 个字符，形成等长并列，Flex 只能退回"选靠前的规则"，于是类别取决于书写顺序——这不是我们想要的语义。写成 `0|[1-9][0-9]*` 后，DEC 在正规式层面就无法匹配"以 0 开头的多位数字"，`075` 只有一个规则能匹配到 3 个字符，歧义在设计阶段就被消除。这是本实验里最值得记住的一条经验：能用正规式表达清楚的地方，不要交给规则顺序去兜底。

终端实录 5-2 主样例：五种输入依次分类

```
$ flex numbers.l && gcc -o numbers.exe lex.yy.c
$ printf '123 0x1F 075 3.14 0\n' | ./numbers.exe
DEC: 123
HEX: 0x1F
OCT: 075
FLOAT: 3.14
DEC: 0
```

与指南要求的"依次输出 DEC、HEX、OCT、FLOAT、DEC"完全一致。最后一行的 `0` 被判为 DEC 而不是 OCT，正是 `0[0-7]+` 要求"至少一位后续数字"的效果。

边界输入：先预测，再运行

指南要求至少自选两组边界输入，且必须先写预测再核对。下面两组我都在纸上写下切分结果后才运行。

表 5-2　第一组边界输入：`08 0x .5 -12 007`

输入	预测	实际输出	是否一致
`08`	DEC 0、DEC 8（OCT 不成立，因 8 不在 0–7）	DEC: 0 / DEC: 8	一致
`0x`	DEC 0、UNHANDLED x（缺十六进制数字）	DEC: 0 / UNHANDLED: x	一致
`.5`	UNHANDLED .、DEC 5（FLOAT 要求左侧有数字）	UNHANDLED: . / DEC: 5	一致
`-12`	UNHANDLED -、DEC 12（负号不属于数字规则）	UNHANDLED: - / DEC: 12	一致
`007`	OCT 007（长度 3 > DEC 的 1）	OCT: 007	一致

表 5-3　第二组边界输入：`0X1f 1. 1.2.3 0000 0x1G 12abc`

输入	预测	实际输出	是否一致
`0X1f`	HEX（大写前缀 + 小写数字位）	HEX: 0X1f	一致
`1.`	DEC 1、UNHANDLED .（右小数点后无数字）	DEC: 1 / UNHANDLED: .	一致
`1.2.3`	FLOAT 1.2、UNHANDLED .、DEC 3	FLOAT: 1.2 / UNHANDLED: . / DEC: 3	一致
`0000`	OCT 0000（首位 0，其余全在 0–7）	OCT: 0000	一致
`0x1G`	HEX 0x1、UNHANDLED G（G 非十六进制位）	HEX: 0x1 / UNHANDLED: G	一致
`12abc`	DEC 12、UNHANDLED a b c	DEC: 12 / UNHANDLED: a / UNHANDLED: b / UNHANDLED: c	一致

十一个边界输入全部预测正确，但结论不能停在"猜对了"。这几组输入真正揭示的是扫掠式识别的本质：

1. **局部匹配不等于整体合法**。`08` 被切成 `0` 和 `8` 两个 DEC，程序"成功识别了"，但 `08` 这个整体既不是合法八进制也不是合法十进制。`12abc` 同理。指南要求"不能把若干局部匹配当作整个数字合法"，指的正是这种情况。**词法分析器只负责切分，合法性判断要靠后继阶段**。
2. **`.5` 与 `1.` 的不对称，是规则设计的结果，不是 bug**。因为我按指南把"简单浮点数"定义为小数点两侧都有数字，所以这两个输入各自只完成一半匹配。如果允许 `.5` 或 `1.` 成立，正规式要改成 `[0-9]*\.[0-9]+` 之类，会与"小数点两侧均有数字"的要求冲突。

3. **负号必须由独立的运算符规则处理。**`-12` 中被切出的 `-` 是 `UNHANDLED`，因为 `numbers.l` 没有定义运算符。若真的加上负号识别，正确做法是让 `-` 成为独立的运算符记号，而不是把它并进数字规则——否则 `a-12` 这类表达式会被切错。

5.4 实验 3：关键字、标识符与整数

`mini_c.l` 要组织三类规则，并按输入顺序连续输出。规则本身很朴素，真正的观察点在规则之间的相互作用。

表 5-4　`mini_c.l` 的规则组织（原始顺序）

顺序	正规式	标记	说明
1	`int|if|while`	KW	关键字，文本固定
2	`[A-Za-z_][A-Za-z_0-9]*`	ID	字母或下划线开头，后接零个或多个字母/数字/下划线
3	`[0-9]+`	INT	十进制整数
4	`[ \t\r\n]+`	—	跳过空白
5	`.`	UNHANDLED	保留未处理字符提示

终端实录 5-3　主样例与最长匹配反例

```
$ flex mini_c.l && gcc -o mini_c.exe lex.yy.c
$ printf 'int integer if ifx while while2 _x 42\n' | ./mini_c.exe
KW: int
ID: integer
KW: if
ID: ifx
KW: while
ID: while2
ID: _x
INT: 42

$ printf 'intx if_ while9 42x\n' | ./mini_c.exe
ID: intx
ID: if_
ID: while9
INT: 42
ID: x
```

主样例的八个单元全部符合预期。`integer` 被整体识别为 `ID` 而不是 `KW: int` 加 `ID: eger`，原因是标识符规则能匹配 7 个字符，关键字规则只能匹配 3 个——**最长匹配优先作用于"所有规则"而不只是"当前规则"**。`intx`、`if_`、`while9` 三个反例进一步说明：只要标识符规则匹配得更长，关键字就抢不走前缀。`42x` 则展示了另一端：`INT` 吃掉 `42`，剩下 `x` 再作为一个 `ID`，没有"数字标识符"这种混合体。

交换规则顺序

把第 1、2 条规则的位置对调，其余不动，重新生成、编译、运行：

表 5-5　规则顺序调整前后的对比

输入	关键字在前	标识符在前	变化原因
`int`	KW: int	ID: int	变了。两条规则都匹配 3 个字符，等长时取靠前者
`integer`	ID: integer	ID: integer	不变。ID 匹配 7 个字符更长，与顺序无关
`if`	KW: if	ID: if	变了，同 `int`
`ifx`	ID: ifx	ID: ifx	不变。ID 匹配 3 个字符，KW 只有 2 个
`while2`	ID: while2	ID: while2	不变，同 `ifx`

终端实录 5-4　交换顺序后 Flex 的额外警告

```
$ win_flex mini_c_swapped.l
mini_c_swapped.l:7: warning, rule cannot be matched
$ printf 'int integer if ifx while while2 _x 42\n' | ./mini_c_swapped.exe
ID: int
ID: integer
ID: if
ID: ifx
ID: while
ID: while2
ID: _x
INT: 42
```

这个 `warning, rule cannot be matched` 是本实验最有价值的副产品：它说明交换之后关键字规则已经彻底不可达——任何能被 `int|if|while` 匹配的文本，都必然也能被前面的标识符规则匹配到同样长度，而等长时后者永远优先。Flex 在生成阶段就能静态发现这一点，这本身就是"规则集合被编译成自动机"的一个证据：只有在自动机层面比较过规则的覆盖关系，才可能给出这种诊断。

5.5 最长匹配与等长规则优先级

Flex 的选择规则可以精确地写成两句：

1. **最长匹配优先**：在所有能匹配当前位置的规则中，选匹配文本最长的那一条。这与规则书写顺序无关。
2. **等长时取靠前者**：若多条规则匹配到同样长的文本，选在 `.l` 文件中出现得更早的那一条。

表 5-6　两条规则与本次实验中的判定实例

规则	本次实验中的实例	若违反这条规则会怎样
最长匹配	`integer` → `ID: integer`（7 > 3）；`ifx` → `ID: ifx`（3 > 2）；`075` → `OCT: 075`（3 > 1）	若按"先匹配到就返回"，`integer` 会被切成 `KW: int` + `ID: eger`，标识符就永远无法被正确识别
等长优先	`int` → `KW: int`（两条规则都是 3 个字符，关键字在写前面）	若反过来把 ID 写在前面，关键字会在编译期全部退化为标识符（即终端实录 5-4 的结果）

指南要求"给出一个自测反例"，我用的是 `intx`：它同时是"关键字前缀 + 后缀"的形式，若实现采用"先试关键字、成功即返回"的策略，就会被切成 `KW: int` 与 `ID: x`；实际输出是 `ID: intx`，说明最长匹配确实在所有规则之间统一比较。另一个方向的反例是 `int` 本身：它无法用长度区分，只能靠顺序——这就是为什么所有 C 风格词法分析器都把关键字规则写在标识符规则之前。

5.6 从正规式到自动机

本实验只要求理解"正规式用于描述词法规则，Flex 借助自动机完成识别"这一层关系，具体构造算法留给项目 2。不过 `lex.yy.c` 里已经能看到不少线索：生成文件中有 `yy_accept[]` 这样一张状态接受表，以及 `yy_current_state` 的状态转移驱动循环。

终端实录 5-5 生成文件中与自动机有关的结构

```
$ grep -n "yy_accept\[" lex.yy.c | head -2
366:static const flex_int16_t yy_accept[20] =
696:            if ( yy_accept[yy_current_state] )
```

这段代码透露的实现方式可以概括为：每条正规式先被转换成 NFA，多条规则合并成一个 NFA，再确定化为 DFA，最后把 DFA 压缩成两张表——转移表与接受表。`yy_accept[state]` 给出"到达该状态时接受哪条规则"，第 696 行那句判断正是"如果当前状态是接受态，就记下它（以便继续往后试更长的匹配）"。这恰好解释了最长匹配是怎么实现的：扫描时不断前进，把最近一次遇到的接受态记下来；走到不能再走时，回退到那个接受态并按对应规则执行动作。所谓"最长"，在实现上就是"最后一次能接受的位置"。

这也解释了为什么 `numbers.l` 里 DEC 写成 `0|[1-9][0-9]*` 比写成 `[0-9]+` 更稳妥：前者让自动机在读完 `0` 之后对"下一位是数字"这件事不再有接受路径，后者会造出一条与 OCT 并行的长路径，两个接受态指向不同规则，最终只能靠规则编号（即书写顺序）打破平局。项目 2 会亲手做 Thompson 构造与子集构造，那时这张表将被完整地推导一遍。

第 6 章　思政融合：国产编译器调研

6.1 调研对象

本次调研选择华为毕昇编译器（BiSheng Compiler）。按 openEuler 软件包仓库的官方描述，它是"华为编译器实验室针对鲲鹏等通用处理器架构场景，打造的一款高性能、高可信及易扩展的编译器工具链"，通过增强和引入多种编译优化技术，为 ARM 服务器平台提供从源码到二进制的完整编译能力[1]。选择它的理由很直接：它是国内少数已经进入主流 Linux 发行版软件源、可以被普通用户 `yum install` 下来直接使用的自研编译器，而不是停留在发布会上的概念产品。

6.2 关键发现

（1）支持语言：C / C++ / Fortran 三线并行

毕昇编译器不自己发明语言，而是复用 LLVM 生态的前端：C 与 C++ 交由 Clang 完成词法、语法、语义分析与 LLVM IR 生成，同时 Clang 兼任驱动程序；Fortran 交由 Flang 完成，官方文档特别指出 Flang 由 `flang1` 与 `flang2` 两个组件构成，同样是"把源码转成 LLVM IR"的驱动[1]。这一点与我在第 2 章看到的 GCC 结构形成了有趣的对照：GCC 的 C 前端 `cc1` 是一个高度集成的单体程序，而 LLVM 系的前端是按库拆分的。

（2）目标架构：以 AArch64 / 鲲鹏为主战场

官方安装文档把"架构"一栏直接写死为 AArch64，软件源路径也是 `repo.oepkgs.net/bisheng/aarch64/`[1]。这说明毕昇的定位不是"通用跨平台编译器"，而是面向 ARM 服务器、特别是鲲鹏处理器的深度定制。鲲鹏社区的文章也印证了这条路线：面向 x86 迁移到鲲鹏的场景，华为提供毕昇编译器、毕昇 JDK、GCC for openEuler 三件套，分别对应高性能计算、Java 业务和通用场景[2]。

（3）工具链组成：驱动 + 前端 + 链接器，全部可替换

编译 C/C++ 用 `clang` / `clang++`，编译 Fortran 用 `flang`；链接默认调用系统 `ld`，但可以通过 `-fuse-ld=lld` 换成 LLVM 自己的 lld[1]。这个细节值得注意——编译器和链接器是解耦的，这与我在 3.6 节用 `-###` 看到的 `collect2 → ld` 是同一套分工思路。安装方式上，openEuler 已把毕昇收进官方源，`yum install bisheng-compiler` 即可；非 openEuler 系统则通过新增 `bisheng-compiler.repo` 的方式接入[1]。

（4）一项有来源支持的技术特征：中后端深度优化 + Auto-tuner

官方文档的表述是："除 LLVM 通用功能和优化外，毕昇编译器的工具链对中端及后端的关键技术点进行了深度优化，并集成 Auto-tuner 特性支持编译器自动调优"[1]。这里有两个可核查的点：其一，优化发生在中端（IR 层）与后端（目标代码层），而不是只在前端做语法层面的把戏——这与第 4 章

观察到的 `-O1` 常量折叠、`-O2` 死代码消除属于同一层级的工作；其二，Auto-tuner 是一种"自动搜索编译参数"的机制，它需要反复编译、运行基准程序并测量，本质上把"人工试 `-O2` 还是 `-O3`、要不要开某个 `-march`"这件事交给机器迭代。这正是第 4 章那张"按场景选选项"表格的自动化版本。

性能方面，鲲鹏社区的对比口径是：在 SPEC CPU 2017 测试中，毕昇编译器相比开源 GCC 9.3 性能提升约 25%，在 WRF、CUACE 等专业应用中提升约 10%[2]。需要说明，这是厂商公布的测法，测试平台、基线与编译选项都以对方口径为准，不宜直接外推到自己手上的程序。

6.3 我的思考：从"用工具"到"想工具"

做完项目 1 再回头看毕昇，最强烈的感受是"四阶段"这套骨架是跨编译器的通用语言。`clang -E`、`clang -S`、`clang -c` 与 GCC 的同名选项语义一致，因为二者都是"驱动程序 + 前端 + 优化器 + 后端 + 汇编器 + 链接器"的结构，只是前端实现方式不同。这意味着我在第 3 章学到的观察方法——用 `-E` 看宏展开、用 `-S` 看指令、用 `nm` 看符号、用 `objdump -r` 看重定位——换成任何一款类 LLVM 的编译器都仍然有效。工具会换，方法论不会。

更值得记住的是差异背后的原因。GCC 的 `cc1` 是单体可执行文件，Clang 是库化的：`libclang` 可以被 IDE、静态分析器、代码补全工具直接调用，所以你会看到 VS Code、CLion 都能读懂 C++ 而不仅仅是"调用编译器"。这解释了为什么 LLVM 生态在工具化、可复用性上扩张得这么快，也解释了国产编译器为什么普遍选择"基于 LLVM 深度定制"而不是"从零写一个 GCC"——**站在成熟的开源基础设施上做增量创新，是更现实的技术路线**。

局限同样清楚。第一，架构绑定：官方文档把架构限定为 AArch64，意味着在 x86 开发机上无法直接使用，迁移成本落在"有没有 ARM 机器"这一步。第二，生态依赖：它需要系统已有的 GCC（≥4.8.5）、glibc（≥2.17）、libstdc++（≥6）、libatomic（≥1.2）作为地基[1]，并不是完全自包含的工具链——这既是优势（与系统 ABI 天然兼容），也意味着它无法脱离 Linux 发行版的包管理体系独立存在。第三，调优的时间成本：Auto-tuner 靠反复编译测量来收敛，对"一天构建几十次"的开发流程并不友好，更适合发布前的最终构建。

关于"一次可复现体验"，我设计了如下步骤，作为后续在鲲鹏环境上的验证方案（本机为 x86_64 Windows，无法直接执行）：在 openEuler AArch64 机器上执行 `yum install bisheng-compiler`，用 `clang -v` 确认版本；然后对项目 1 的 `hello.c` 分别执行 `clang -E`、`clang -S`、`clang -c`，把三份产物与本次 GCC 的 `hello.i`、`hello.s`、`hello.o` 并排比较：预期会在 `.i` 的行标记格式、`.s` 的伪指令（GCC 用 `.seh_*`，Clang 风格不同）、`.o` 的节名与符号修饰上看到差异，但四个阶段的划分、`nm` 中 `T`/`U` 的含义、`objdump -r` 的重定位项应当保持一致。如果这三条一致，就说明"四阶段"确实是工具链层面的客观结构，而不是某一家编译器的实现细节。

还有一点体会来自"编译器为什么是基础软件"。第 4 章里，同一段循环在 `-O0` 下要跑 11 ms，在 `-O2` 下被彻底消除——编译器不动一行源码，就能决定程序的性能上限。把这个尺度放大到一个数据中心、一座超算集群，编译器的差距就是电费、机时和能耗的差距。这大概就是"卡脖子"三个字在编译领域的具象含义：它不在应用层，而在所有人都站在上面的那一层。

6.4 参考文献

[1] openEuler OEPKGS. huawei/bisheng（毕昇编译器仓库说明）[EB/OL]. https://docs.oepkgs.net/zh-CN/Repos/huawei-bisheng/，访问日期 2026-10-25.

[2] 鲲鹏社区. GCC for openEuler 与鲲鹏编译工具链介绍[EB/OL]. https://www.hikunpeng.com.cn/forum/thread-0225203517001409120-1-1.html，访问日期 2026-10-25.

[3] LLVM Project. Clang Compiler User's Manual[EB/OL]. https://clang.llvm.org/docs/UsersManual.html，访问日期 2026-10-25.

[4] GCC Team. Overall Options (Using the GNU Compiler Collection)[EB/OL]. https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html，访问日期 2026-10-25.

第 7 章　综合反思

7.1 最有趣的环节：`-O1` 留下的那个空循环

如果只能留一个发现，我会留 4.1 节那个：`-O1` 已经把 `sum` 折叠成常量 `100000000000000`，却仍然保留了 `subq $1, %rax / jne .L2` 这样一对空的计数指令，导致"算术都不做了，计时还有 3 毫秒"。它推翻了我原先"优化等级是一个单调的滑杆"的直觉——真实情况是一组彼此独立的变换，各自有不同的触发条件。常量折叠在 `-O1` 就会发生，而死循环消除要等到 `-O2`。这也让我第一次具体地理解了"半优化状态"为什么不安全：既丢了可调试性，又没拿到性能。

7.2 最大的收获：把"编译"从一个词拆成了一条流水线

做实验之前，"编译"对我是一个不可分割的动作；做完之后，它变成了五样东西、四个阶段、两个关键机制（符号解析与重定位）。最实用的三个收获是：

1. **出错时先判断"错在哪个阶段"**。语法错带行列号、发生在编译期；`undefined reference` 发生在链接期，报的是目标文件名而不是源文件名。这个区分能省掉大量无谓的排查——我原来会去检查 `#include`，现在会先看它是 `.c` 还是 `.o`。
2. **`-###` 是理解工具链最快的入口**。与其去背"四阶段"，不如看一眼它实际调了哪几个程序、临时文件怎么命名。这一条对以后接触任何新编译器都适用。
3. **能交给正规式解决的歧义，不要留给规则顺序**。5.3 节把 DEC 写成 `0|[1-9][0-9]*` 而不是 `[0-9]+`，效果是把"`075` 算 DEC 还是 OCT"这个顺序依赖问题在源头消除了。这条经验不只用于 Flex。

7.3 还存在的疑问

1. 为什么 `-O2` 的可执行文件反而比 `-O0` 大 36 字节？代码段明明更小（11 324 < 11 420）。我推测是 PE 的节对齐（FileAlignment / SectionAlignment）与重定位、导入表布局变化造成的，但没有逐节比对 `objdump -h` 的文件偏移，还不能下结论。
2. `clock()` 在本机的分辨率到底是多少？`-O0` 的九次采样从 0.008 s 波动到 0.017 s，跨度接近一倍。`CLOCKS_PER_SEC` 在 Windows 上定义为 1000，但底层时钟源的实际粒度、以及它是否受 CPU 频率调节影响，我没有验证。`-O2` 的九次全 0 说明粒度至少不劣于 1 ms，但"1 ms 以下的东西测不出来"这件事本身需要确认。
3. Flex 生成的表是怎么压缩的？生成文件里 `yy_accept[20]` 只有 20 项，而规则有 5 条、状态显然不止 20 个——说明状态编号经过了重新映射，还可能用了等价类合并（把输入字符分成若干等价类以减少表宽）。这部分属于 DFA 最小化与表压缩，项目 2 应该会展开。
4. 如果把 `hello.c` 换成 Clang 编译，四阶段的产物差异会有多大？这是第 6 章那份验证方案要回答的问题，目前只有推断，没有实测。

7.4 关于实验方法的一点体会

这次实验里我最刻意坚持的一件事，是凡是"预期"和"实测"不一致的地方，都写清楚不一致本身。比如指南说 `file hello` 应显示 `ELF`，我的机器显示 `PE32+`；指南说缺 `main` 会报 `undefined reference to 'main'`，我的机器报 `WinMain`。把这些差异当成"实验结果的一部分"记录，比强行对齐预期更有价值——它们恰好说明四阶段是抽象结构，而格式与命名是实现细节。同样地，五类警告我没有照抄指南的行号，而是记录本机实际报出的 `2:28`、`4:9` 等位置；优化对比表填的是九次采样的中位数，而不是单次结果。这些做法让报告里的每个数字都可以被复现。

7.5 任务完成情况自评

对照任务书的 6 个任务逐项核对，全部必做项均已完成，且每一项都留下了可复现的记录：

表 7-1　任务完成情况

任务	要求	状态	对应本报告位置
1.0	环境配置与自检	完成	第 1 章：版本表 1-1、自检记录 1-1/1-2
1.1	编译、读错误、多文件	完成	第 2 章：三个验收点见表 2-1
1.2	四阶段产物与 `-###`	完成	第 3 章：表 3-1、表 3-2、表 3-3
1.3	四个编译选项实验	完成	第 4 章：表 4-1（9 次采样）、表 4-2（5 类警告）、4.3/4.4
1.4	三个 Flex 实验	完成	第 5 章：`numbers.l` 与 `mini_c.l` 自行完成，两组边界输入见表 5-2/5-3，顺序对比见表 5-5
1.5	国产编译器调研 800 字以上	完成	第 6 章：约 1 500 字，参考文献 4 条（含 2 条官方资料）

仍有的不足也说清楚：其一，实验平台是 MinGW-w64 而非指南首选的 Linux/WSL，虽然四阶段机制一致，但 ELF 相关的观察（如 `readelf` 的段表、`.rela.text` 的具体形式）没有亲手做；其二，`-O2` 之后代码段不再变化，我没有进一步用 `objdump -d` 逐条比对指令数，只用了行数与节大小作为代理指标；其三，毕昇编译器的调研停留在文档层面，缺少在 AArch64 实机上的验证。这三点都可以作为后续补充的方向。

附录 A　源文件清单与关键源码

表 A-1　提交的源文件

序号	文件	对应任务	作用
1	`hello.c`	1.1 / 1.2	最简编译流程与四阶段产物观察的被测程序
2	`bug.c`	1.1	故意缺右括号与分号，用于观察语法诊断（保留，不修复）
3	`bug_fixed.c`	1.1	`bug.c` 的修复版，补充 `)`、`;` 与换行
4	`add.c`	1.1 / 1.2	提供 `add` 定义的独立编译单元
5	`main.c`	1.1 / 1.2	通过 `extern` 声明调用 `add`，用于复现链接错误
6	`loop.c`	1.3 实验 1	循环密集型计时程序，`N = 10 000 000`
7	`warn.c`	1.3 实验 2	含 5 类缺陷（未定义行为）的诊断样本，仅编译不运行
8	`warn_fixed.c`	1.3 实验 2	`warn.c` 的修复版，`-Wall -Wextra -Werror` 下零诊断
9	`test.c`	1.3 实验 3/4	用于对比 `-g` 与 `-DDEBUG` 的小程序
10	`demo.l`	1.4 实验 1	最简整数识别器（工具示例，原样运行）
11	`numbers.l`	1.4 实验 2	自行完成：四类数字字面量规则
12	`mini_c.l`	1.4 实验 3	自行完成：关键字、标识符、整数三类规则

A.1 `warn.c` 与修复版 `warn_fixed.c`

```
/* warn.c — 仅用于编译诊断，含未定义行为，禁止链接运行 */
#include <stdio.h>
int missing_return(void) { }              /* ① -Wreturn-type        */
int main(void) {
    int unused = 0;                        /* ② -Wunused-variable    */
    int x;                                 /* ⑤ 未初始化             */
    int y = 0;
    if (y = 5) printf("y=%d\n", y);        /* ③ -Wparentheses        */
    printf("%d\n", "wrong type");          /* ④ -Wformat=            */
    return x;
}
```

```c
/* warn_fixed.c — 同样选项下零警告 */
#include <stdio.h>

int missing_return(void) { return 0; }

int main(void) {
    int x = 0;
    int y = 0;
    if (y == 5) printf("y=%d\n", y);
    printf("%d\n", 42);
    (void)x;
    return 0;
}
```

A.2 `numbers.l`（自行完成部分）

```
%option noyywrap
%{
#include <stdio.h>
%}
%%
0[xX][0-9a-fA-F]+   { printf("HEX: %s\n", yytext); }
[0-9]+\.[0-9]+      { printf("FLOAT: %s\n", yytext); }
0[0-7]+             { printf("OCT: %s\n", yytext); }
0|[1-9][0-9]*       { printf("DEC: %s\n", yytext); }
[ \t\r\n]+          { /* skip whitespace */ }
.                   { printf("UNHANDLED: %s\n", yytext); }
%%
int main(void) { yylex(); return 0; }
```

A.3 `mini_c.l`（自行完成部分）

```
%option noyywrap
%{
#include <stdio.h>
%}
%%
int|if|while           { printf("KW: %s\n", yytext); }
[A-Za-z_][A-Za-z_0-9]* { printf("ID: %s\n", yytext); }
[0-9]+                 { printf("INT: %s\n", yytext); }
[ \t\r\n]+             { /* skip whitespace */ }
.                      { printf("UNHANDLED: %s\n", yytext); }
%%
int main(void) { yylex(); return 0; }
```

附录 B　完整命令清单

以下命令按任务顺序排列，均在 `build/` 目录（或调试用的 `C:\temp\clab`）下执行。`$` 为提示符，不属于命令。

表 B-1　全部实验命令

任务	命令
1.0	`gcc -Wall -Wextra -O0 -g check.c -o check.exe`；`./check.exe`；`nm check.exe | grep -E " (T|t) (main|add)$"`；`objdump -d check.exe`；`ldd check.exe`；`flex demo.l && gcc -o demo lex.yy.c`；`printf '123 hello 456\n' | ./demo`
1.1	`gcc hello.c -o hello.exe`；`./hello.exe`；`file hello.exe`；`gcc bug.c -o bug.exe`；`gcc bug_fixed.c -o bug_fixed.exe`；`gcc main.c -o broken.exe`；`gcc main.c add.c -o two_files.exe`
1.2	`gcc -E hello.c -o hello.i`；`wc -l hello.i`；`head -10 hello.i`；`grep -c '^#' hello.i`；`gcc -E -P hello.c -o hello_no_markers.i`；`gcc -S hello.i -o hello.s`；`grep -n -A 6 -E '^_?main:' hello.s`；`gcc -c hello.s -o hello.o`；`file hello.o`；`nm hello.o`；`objdump -h hello.o`；`objdump -r hello.o`；`gcc hello.o -o hello_from_o.exe`；`ldd hello_from_o.exe`；`gcc -c add.c -o add.o`；`gcc -c main.c -o main.o`；`gcc add.o -o no_main.exe`；`gcc main.o add.o -o two_files_o.exe`；`gcc -### hello.c -o hello_trace 2> trace.txt`
1.3-1	`gcc -O{0,1,2,3} loop.c -o loop_O{0,1,2,3}.exe`；`gcc -O{0,1,2,3} -S loop.c -o loop_O{0,1,2,3}.s`；`wc -c`；`wc -l`；`size`；`time ./loop_O0.exe`；`gcc -Q --help=optimizers -O0`（其余档位同理）
1.3-2	`gcc -O1 -Wall -Wextra -c warn.c -o warn.o`；`gcc -O1 -Wall -Wextra -Werror -c warn.c -o warn.o`；`gcc -O1 -Wall -Wextra -Werror -c warn_fixed.c -o warn_fixed.o`
1.3-3	`gcc -O0 -g test.c -o test_with_gdb.exe`；`gcc -O0 test.c -o test_no_gdb.exe`；`gdb --batch -x gdbcmds.txt ./test_with_gdb.exe`（脚本内容：`break main` / `run` / `next` / `next` / `print x` / `info locals` / `quit`）；`objdump -h test_with_gdb.exe | grep debug`
1.3-4	`gcc -O0 test.c -o test_off.exe`；`gcc -O0 -DDEBUG test.c -o test_on.exe`；`strings test_on.exe | grep -c "\[DEBUG\]"`
1.4-1	`flex demo.l`；`gcc -o demo.exe lex.yy.c`；`printf '123 hello 456\n' | ./demo.exe`
1.4-2	`flex numbers.l`；`gcc -o numbers.exe lex.yy.c`；`printf '123 0x1F 075 3.14 0\n' | ./numbers.exe`；`printf '08 0x .5 -12 007\n' | ./numbers.exe`；`printf '0X1f 1. 1.2.3 0000 0x1G 12abc\n' | ./numbers.exe`
1.4-3	`flex mini_c.l`；`gcc -o mini_c.exe lex.yy.c`；`printf 'int integer if ifx while while2 _x 42\n' | ./mini_c.exe`；`printf 'intx if_ while9 42x\n' | ./mini_c.exe`；交换规则顺序后：`flex mini_c_swapped.l` 并重复上述输入

— 报告完 —